Freight Rate Prediction — Machine Learning Engineer Assessment

Author: Shahd Ghunimah

1. Summary

A LightGBM model trained on log(posted_rate) predicts the 12,000 Nov–Dec 2025 validation loads. Validation uses time-based backtests that mirror the real task (train on everything up to the first month of a quarter, forecast the next two months). The final model reaches 1.78% and 1.67% MAPE (MAE $43 / $38) on clean labels in the two backtest folds, against 8–9% for a rate-per-mile baseline and 3–4.5% for a log-linear regression.

2. Key findings from the data

- **Structure.** Distance explains most of the rate (rate ∝ distance^0.87). Reefer is about 13% and Flatbed about 8% more expensive than Dry Van, and heavier loads cost more.
- **Quarter-end seasonality.** Rates ramp up 5–6% in the last ~5 weeks of every quarter, then reset on the 1st of the next quarter. Each quarter also sits about 1.5–2% above the previous one. Nov–Dec is the end of Q4, so the forecast has to capture this ramp.
- **Market index.** market_index is a daily market signal with a strong weekly cycle (peaks Wed/Thu). It is lower in Nov–Dec (~0.93) than the training average (1.08), but still inside the training range.
- **Unseen cities.** 8 validation cities never appear in training (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego), about 12% of validation loads. The model therefore uses coordinates, not city IDs.

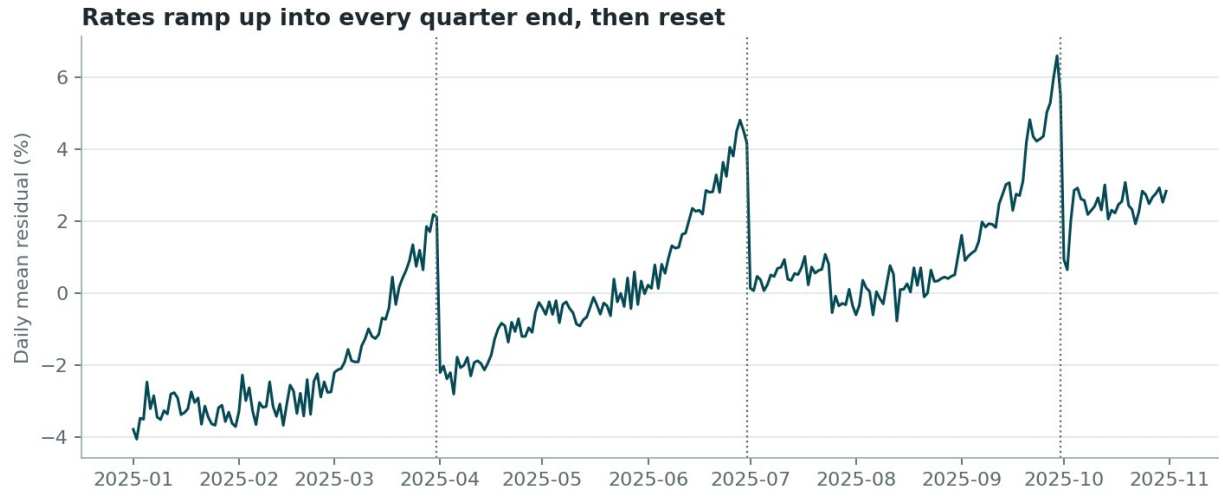


Figure 1 – Daily mean residual of a simple log-linear model: rates ramp into each quarter end.

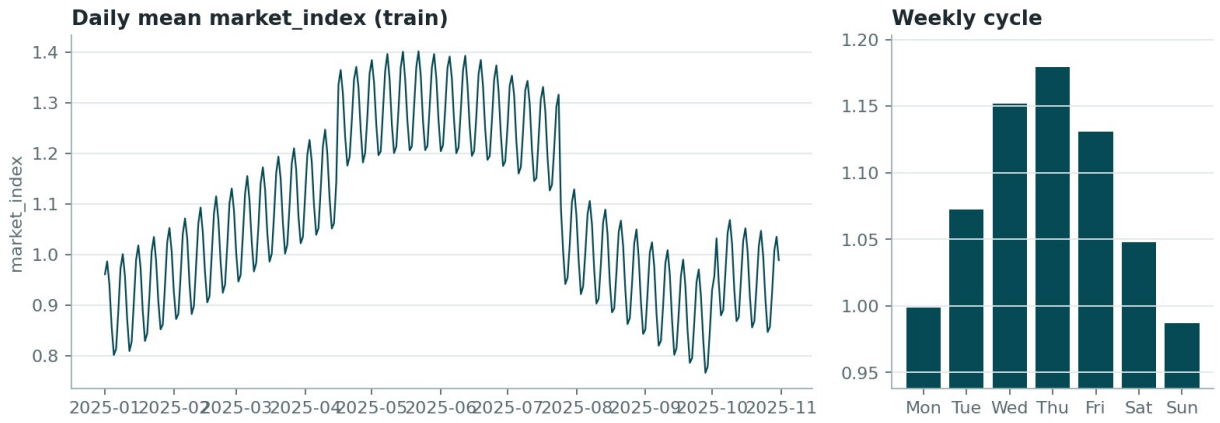


Figure 2 – market_index over time and its weekly cycle.

3. Data-quality issues and how they were handled

Issue	Train	Validation	Handling
Corrupted labels (rate ×0.17–0.45 or ×2.2–4.6)	677 (1.4%)	n/a	Flagged by an iteratively refit robust log-linear model (|log residual| > 0.4) and excluded from training
Negative weight	292	145	Sign error, so absolute value taken
Missing weight	300	165	Median weight of the same equipment type (training statistics)
Missing market_index	374	249	Mean market_index of the same date (within-day std 0.025 vs 0.17 across days)
Weight capped at 47,500 lb	1204	301	Kept: it is a plausible legal/load cap and tree models handle it
Synthetic-looking coordinates	—	—	Coordinates are consistent per city, so they are used as-is. Distance is consistent per lane and is used as the main feature

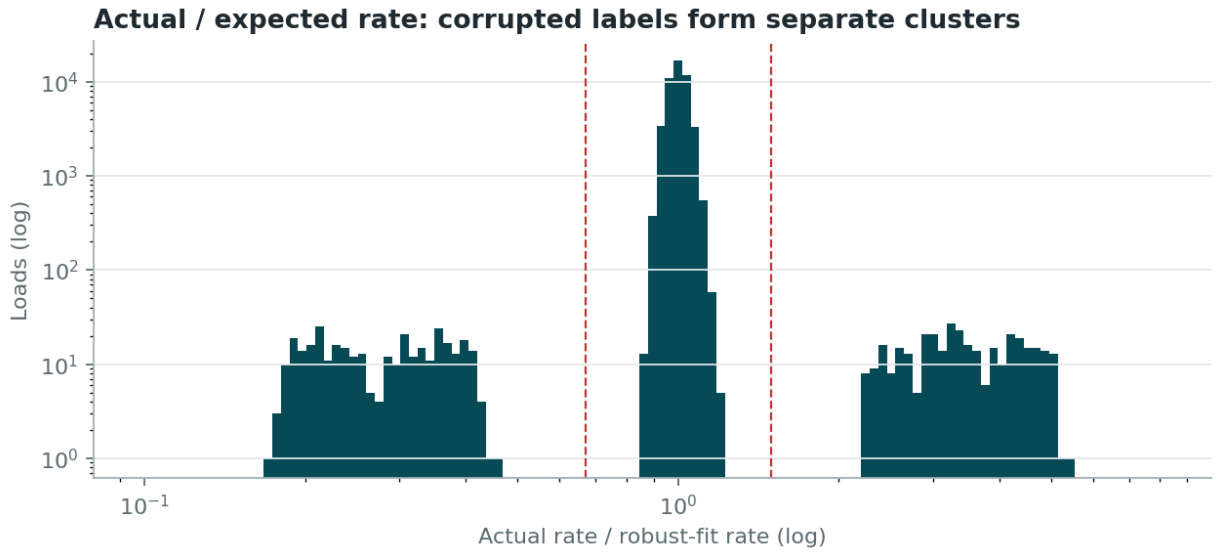


Figure 3 – Actual / expected rate. Corrupted labels form two separate clusters, far from the clean peak. Dashed lines = removal threshold.

4. Train / test split and validation approach

The labelled data covers Jan–Oct 2025 and the loads to predict are Nov–Dec 2025, so this is a forecasting problem. A random split would leak the market conditions of every day into training and give an over-optimistic score. So I used forward-in-time backtests that copy the real setup:

- **Fold A:** train 2025-01-01 .. 2025-04-30 (18,835 clean loads), test 2025-05-01 .. 2025-06-30 (9,696 loads).
- **Fold B:** train 2025-01-01 .. 2025-07-31 (33,253 clean loads), test 2025-08-01 .. 2025-09-30 (9,429 loads).

In both folds the model sees only the first month of the target quarter, exactly like Oct → Nov–Dec. Cleaning statistics and outlier flags are re-fit inside each fold's training window, so no information from the test window leaks in. Metrics are reported on clean labels (the real target) and on all labels. Hyper-parameters (tree size, number of rounds) were chosen on these folds. The final model is then retrained on all clean Jan–Oct data.

Two extra checks:

- **Random split (for comparison only):** a random 80/20 split gives 1.54% MAPE, lower than the honest time-based estimate. This confirms that a random split is optimistic.
- **Unseen-city test:** 8 cities were removed entirely from Fold B's training data. Loads touching them scored 1.71% MAPE vs 1.68% for the other loads, so the model generalises to new cities.

5. Model

LightGBM gradient boosting on log(posted_rate), averaged over 3 seeds. Features: log distance, equipment, weight, market_index, quote_signal, pickup/delivery coordinates, bearing, and calendar features (quarter, days to quarter end, day of week, day of month). The log target turns the multiplicative price structure into an additive one and optimises relative error.

The key design choice is LightGBM interaction constraints: each calendar feature is kept in its own group, so its effect is additive in log space (a multiplicative uplift on the rate). The quarter-end ramp learned in Q1–Q3 then transfers cleanly to the end of Q4. Without the constraint, trees mix the quarter with days-to-quarter-end and cannot extrapolate the ramp (2.62% vs 1.78% MAPE in Fold A). Small trees (4 leaves, 3,000 rounds at learning rate 0.03) generalised better than deeper ones.

6. Results

Model	Fold A MAPE	Fold A MAE	Fold B MAPE	Fold B MAE	Fold B R^2
Baseline: median rate-per-mile x distance	8.93%	$153.7	8.15%	$186.7	0.9577
Baseline: log-linear regression	3.15%	$73.5	4.49%	$106.8	0.9875
LightGBM (unconstrained time features)	2.62%	$58.5	1.70%	$38.0	0.9984
LightGBM + additive calendar effects (final)	1.78%	$43.4	1.67%	$37.7	0.9984

Including the corrupted labels in the test set, the final model's MAE is $97 / $88. The difference comes entirely from labels that are off by a factor of 2–5, which no model can or should predict.

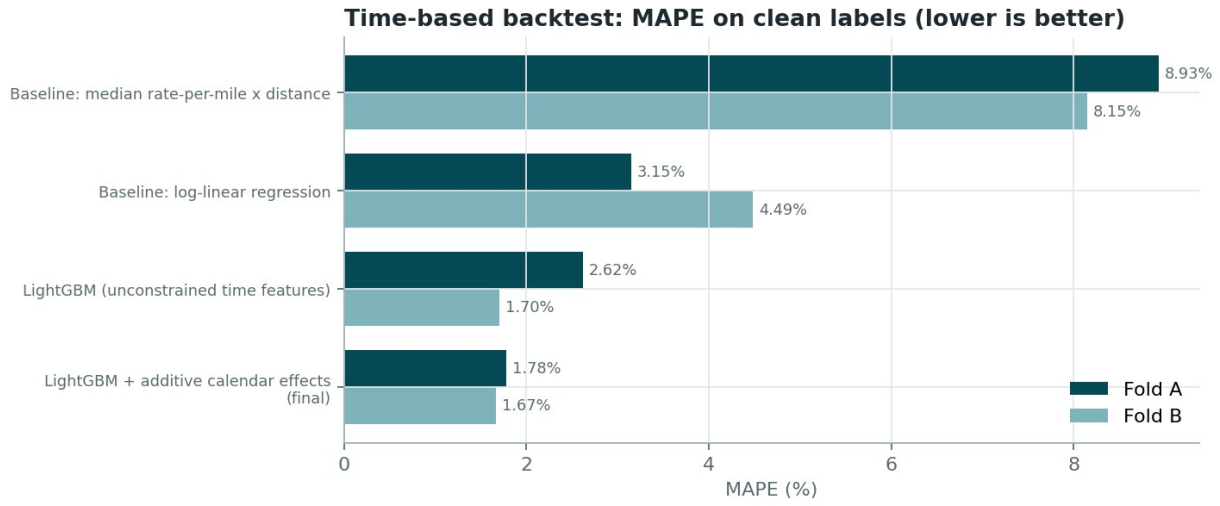


Figure 4 – Backtest MAPE on clean labels.

7. December prediction chart (score.py)

Fixed inputs: Lexington → Fort Wayne, 360 mi, Dry Van, 32,000 lb. Only the date changes. The chart input file has no market signals, so for each day I used the daily mean market_index and median quote_signal observed in validation.csv for that date. The curve rises about 6% through December (the Q4 quarter-end ramp), with a weekly ripple from the market_index cycle.

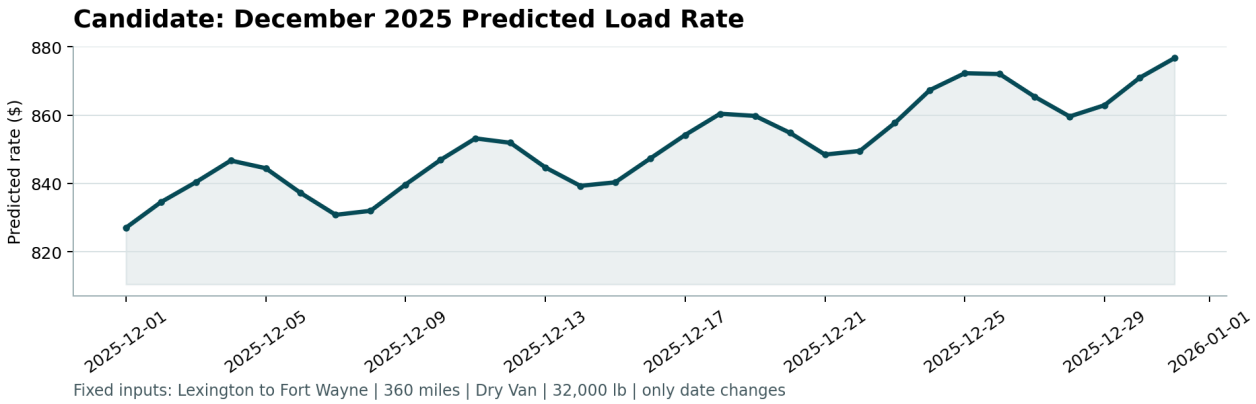


Figure 5 – Output of the provided score.py.

8. Limitations and next steps

- **Holidays:** The training data contains no Thanksgiving or Christmas periods, so holiday effects cannot be learned.
- **Seasonality assumption:** The quarter-end ramp is assumed to behave in Q4 as in Q1–Q3. This matched well in both backtests.
- **Next:** Add prediction intervals (quantile LightGBM) and monitor the corrupted-label rate in production.